

2021数算第二章作业_答案

1. 已知一顺序表 A 有 n 个元素，其元素值递减有序排列，编写一个算法删除顺序表中多余的值相同的元素。要求：空间复杂度为 $O(1)$ ，并分析代码的时间复杂度。

解：

用一个变量记录当前拷贝位置 p ，遍历 A ，若其值与前一个不相同，则将这个值赋予位置 p ，并使得 $p++$ 。

时间复杂度 $O(n)$ ，空间复杂度 $O(1)$

C++  复制代码

```
1  int p = 1;
2  for (int i = 1; i < A.size(); i++){
3      if (A[i] != A[i-1]){
4          A[p] = A[i];
5          p++;
6      }
7  }
```

2. 写出将单链表置逆的算法，输入一个链表的头结点，反转该链表并输出反转后链表的头结点。要求尽可能少地使用附加单元，并分析算法的时间和空间复杂度。

解：

定义三个指针，分别指向当前遍历结点、前一个结点和后一个结点

时间复杂度 $O(n)$ ，空间复杂度 $O(1)$

```
1  ListNode* ReverseList(ListNode* pHead){
2      ListNode* pReversedHead = NULL;
3      ListNode* pNode = pHead;
4      ListNode* pPrev = NULL;
5
6      while(pNode != NULL){
7          ListNode* pNext = pNode->next;
8
9          if(pNext == NULL){
10             pReversedHead = pNode;
11         }
12
13         pNode->next = pPrev;
14         pPrev = pNode;
15         pNode = pNext;
16     }
17     return pReversedHead;
18 }
```

3. 请设计算法判断一个单向链表 L 是否有环，并分析你所设计算法的时间复杂度和空间复杂度。要求：不允许修改给定的链表。

解：

使用快慢指针，快指针每次走两步，慢指针每次走一步。有环必相遇，快指针走到链尾无环。

空间复杂度：两个指针 $O(1)$ ；时间复杂度： $O(n)$

```
1  ListNode* isLoopList(ListNode* pHead){
2      ListNode* slowPointer, fastPointer;
3
4      //使用快慢指针，慢指针每次向前一步，快指针每次两步
5      slowPointer = fastPointer = pHead;
6      while(fastPointer != null && fastPointer->next != null){
7          slowPointer = slowPointer->next;
8          fastPointer = fastPointer->next->next;
9
10         //两指针相遇则有环
11         if(slowPointer == fastPointer){
12             return true;
13         }
14     }
15     return false;
16 }
```

暴力方法，从头节点，依次遍历单链表的每一个节点，每到一个新的节点就头节点重新遍历之前的所有的节点，对比此时的节点，如果相同，证明该节点遍历过两次，以此说明链表是有环的

时间复杂度 $O(n^2)$ ，空间复杂度 $O(1)$